UNISENAI
RAYSSA BORGES DE ALMEIDA

Projeto Coelho

Mensageria Assíncrona

TUBARÃO

2026

1. Visão Geral

O Projeto Coelho é um microsserviço produtor de mensagens desenvolvido com Spring Boot e RabbitMQ.

Em vez de processar pedidos, preços e estoques de forma síncrona (o chamador espera a resposta), o projeto publica mensagens em filas específicas do RabbitMQ. Consumidores independentes leem essas mensagens e as processam em seu próprio ritmo, sem bloquear quem publicou.

1.1 Contexto de Negócio

Uma empresa de e-commerce precisa tornar o processamento de pedidos mais escalável e resiliente. Atualmente, o sistema processa tudo de forma síncrona, o que causa lentidão em horários de pico. Para resolver isso, a empresa decidiu adotar uma arquitetura baseada em mensageria com o RabbitMQ.

Novo fluxo:

1. O cliente realiza um pedido
2. O sistema registra a solicitação
3. Uma mensagem é enviada para uma fila
4. (Futuro) Outros serviços irão consumir essa mensagem

Quando um cliente finaliza uma compra:

- O serviço de pedidos precisa registrar o pedido (fila PEDIDO)
- O serviço de estoque precisa baixar os itens vendidos (fila ESTOQUE)
- O serviço de preços pode precisar ajustar promoções (fila PRECO)

Com mensageria assíncrona, todos esses passos acontecem de forma desacoplada: o sistema não trava enquanto aguarda cada serviço terminar sua tarefa.

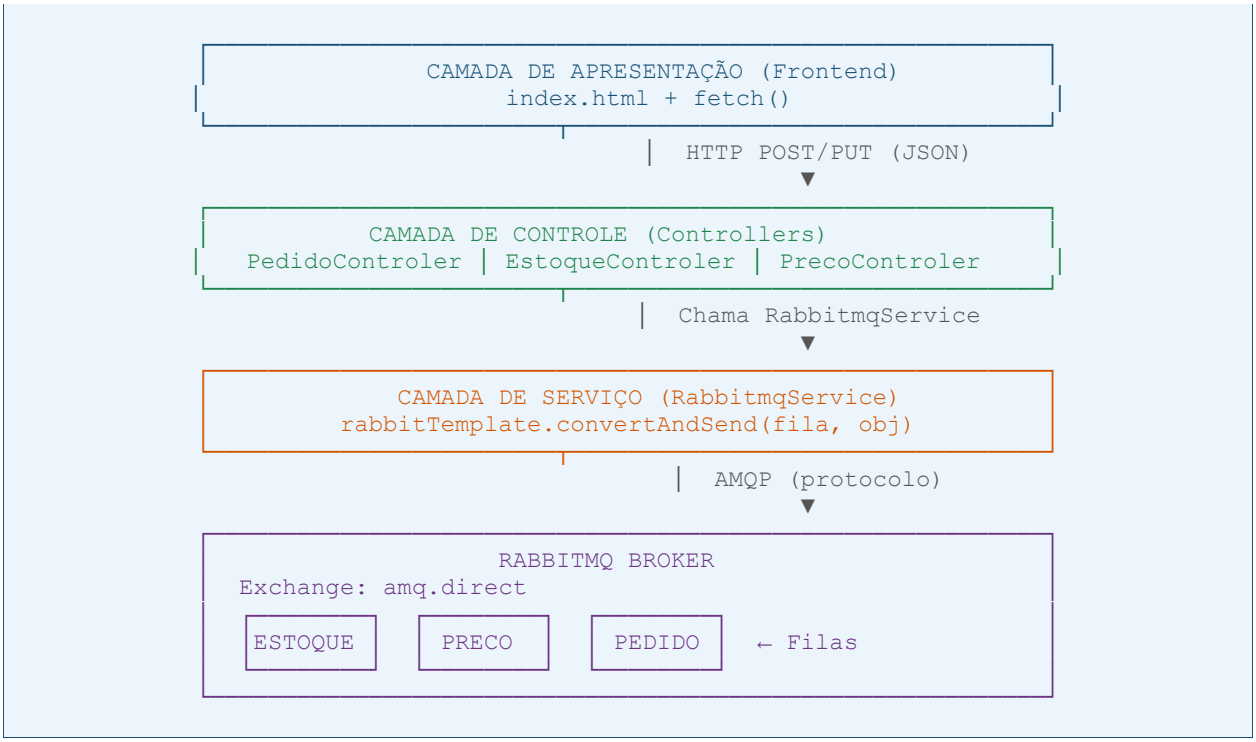
1.2 Tecnologias Utilizadas

Tecnologia	Versão	Papel no Projeto
Java 17+	LTS	Linguagem de programação principal
Spring Boot	3.x	Framework para criação do microsserviço REST
Spring AMQP	3.x	Integração com RabbitMQ via templates e anotações
RabbitMQ	3.x	Broker de mensagens — recebe, armazena e entrega mensagens
Docker	Opcional	Execução do RabbitMQ em container isolado
Maven	3.x	Gerenciamento de dependências e build
HTML / CSS / JS	Vanilla	Interface web simples para envio de pedidos

2. Arquitetura do Sistema

2.1 Diagrama de Componentes

O projeto é um Produtor (Producer). Ele recebe requisições HTTP e as transforma em mensagens que são publicadas no RabbitMQ.



2.2 Fluxo de Dados (passo a passo)

- O usuário preenche o formulário HTML e clica em Enviar
- O navegador executa fetch() e envia uma requisição HTTP ao backend (porta 8087)
- O Controller recebe o JSON e desserializa no DTO correspondente
- O Controller chama RabbitmqService.enviarMensagem(nomeFila, objeto)
- O RabbitTemplate serializa o objeto e o publica no exchange amq.direct
- O RabbitMQ roteia a mensagem para a fila correta via routing key (= nome da fila)
- A mensagem fica na fila aguardando um Consumidor lê-la (fora deste projeto)

3. Estrutura de Pacotes

O projeto segue a estrutura padrão do Spring Boot com separação por responsabilidade:

```
com.example.coelho/
├── CoelhoApplication.java
├── constantes/
│   └── RabbitMQConstantes.java
├── conections/
│   └── RabbitMQConection.java
├── service/
│   └── RabbitmqService.java
├── DTO/
│   ├── PedidoDTO.java
│   ├── EstoqueDTO.java
│   └── PrecoDTO.java
└── controler/
    ├── PedidoControler.java
    ├── EstoqueControler.java
    └── PrecoControler.java
```

Pacote	Responsabilidade
constantes/	Define strings constantes para nomes de filas, evitando uso de strings literais espalhadas pelo código
conections/	Declara as filas, o exchange e os bindings no RabbitMQ ao iniciar a aplicação
service/	Contém a lógica de publicação de mensagens; isola o uso do RabbitTemplate dos controllers
DTO/	Classes de transferência de dados; mapeiam o JSON recebido da requisição HTTP para objetos Java
controler/	Recebe as requisições HTTP, extrai o DTO do body e delega ao serviço de mensageria

4. Configuração do Ambiente

4.1 application.properties

Arquivo localizado em src/main/resources/application.properties:

```
spring.application.name=coelho
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=insira_seu_login_aqui
spring.rabbitmq.password=insira_sua_senha_aqui
server.port=8087
```

Propriedade	Valor Padrão	Descrição
spring.rabbitmq.host	localhost	Endereço do broker RabbitMQ
spring.rabbitmq.port	5672	Porta AMQP padrão do RabbitMQ
spring.rabbitmq.username	insira_seu_login_aqui	Usuário padrão de desenvolvimento
spring.rabbitmq.password	insira_sua_senha_aqui	Senha padrão de desenvolvimento
server.port	8087	Porta onde o Spring Boot expõe a API REST

4.2 Subindo o RabbitMQ com Docker

Execute o comando abaixo para iniciar o RabbitMQ com painel de administração:

```
docker run -d --name rabbitmq \
  -p 5672:5672 -p 15672:15672 \
  rabbitmq:3-management
```

Após a execução, o painel de administração estará disponível em http://localhost:15672

5. Documentação das Classes

5.1 CoelhoApplication.java

Classe principal da aplicação. É o ponto de entrada do programa.

Item	Detalhes
Anotação principal	@SpringBootApplication
Responsabilidade	Inicializa o contexto Spring, faz o auto-scan de componentes e sobe o servidor embutido
Por que usar?	Combina @Configuration + @EnableAutoConfiguration + @ComponentScan em uma única anotação conveniente

5.2 RabbitMQConstantes.java

Classe utilitária que centraliza os nomes das filas como constantes estáticas.

Constante	Valor	Finalidade
FILA_ESTOQUE	"ESTOQUE"	Nome da fila para mensagens de estoque
FILA_PRECO	"PRECO"	Nome da fila para mensagens de preço
FILA_PEDIDO	"PEDIDO"	Nome da fila para mensagens de pedido

Por que usar constantes?

Evitar 'magic strings' (strings literais repetidas no código). Se o nome de uma fila mudar, basta alterar em um único lugar.

5.3 RabbitMQConection.java

Componente Spring responsável por declarar a infraestrutura de mensageria no RabbitMQ no momento em que a aplicação sobe.

Elemento	Detalhes
@Component	Registra a classe como bean gerenciado pelo Spring (será instanciada automaticamente)
@PostConstruct	Garante que o método adiciona() seja chamado logo após o bean ser criado e as dependências injetadas
AmqpAdmin	Interface Spring que permite gerenciar a infraestrutura do broker: criar filas, exchanges e bindings
Queue(nome, true, false, false)	Cria fila durável (sobrevive ao reinício do broker), não exclusiva, não auto-deletável
DirectExchange	Tipo de troca que roteia mensagens pela routing key exata

Binding	Liga uma fila a um exchange usando a routing key (neste projeto, igual ao nome da fila)

5.4 RabbitmqService.java

Serviço que encapsula a publicação de mensagens no RabbitMQ.

Elemento	Detalhes
@Service	Marca como bean de serviço; semanticamente indica camada de negócio
@Autowired RabbitTemplate	Injeta o template de comunicação com o RabbitMQ (configurado automaticamente pelo Spring AMQP)
convertAndSend(fila, obj)	Serializa o objeto (usando Java Serialization ou Jackson, dependendo da config) e envia à fila

Por que criar um Service separado?

Seguindo o princípio da responsabilidade única (SRP), os controllers não precisam saber como se comunica com o RabbitMQ. Se trocar o broker futuramente, só este arquivo muda.

5.5 DTOs (Data Transfer Objects)

Os DTOs são classes simples de transferência de dados. Elas recebem os dados enviados pelo cliente HTTP e os carregam até o serviço de mensageria.

PedidoDTO

Campo	Tipo	Descrição
id	String	Identificador único do pedido
cliente	String	Nome ou código do cliente
valor	BigDecimal	Valor total do pedido (precisão decimal)

EstoqueDTO

Campo	Tipo	Descrição
codigoproduto	String	Código único do produto
quantidade	int	Quantidade a ser ajustada no estoque

PrecoDTO

Campo	Tipo	Descrição

codigoproduto	String	Código único do produto
preco	String	Novo preço do produto (como texto)

Todos os DTOs implementam Serializable. Por quê?

O RabbitMQ precisa converter o objeto Java em bytes para transmissão. A interface Serializable permite que o Java faça essa conversão automaticamente.

5.6 Controllers

Os controllers são o ponto de entrada das requisições HTTP. Cada um gerencia um recurso específico.

PedidoControler — @PostMapping /pedido

Elemento	Detalhes
@RestController	Combina @Controller + @ResponseBody: todo retorno é automaticamente serializado como JSON
@RequestMapping("/pedido")	Prefixo de rota para todos os endpoints desta classe
@PostMapping	Mapeia requisições HTTP POST para este método
@RequestBody PedidoDTO	Spring desserializa o JSON do body da requisição para o objeto PedidoDTO
ResponseEntity<Map<String,String>>	Permite controlar o status HTTP e o body da resposta
Map.of("status", "sucesso", ...)	Retorna um JSON simples confirmando o envio do pedido à fila

EstoqueControler — @PutMapping /estoque

Elemento	Detalhes
@PutMapping	Mapeia requisições HTTP PUT (semanticamente: atualização de recurso)
ResponseEntity<Void>	Retorna HTTP 200 sem body — apenas confirma que a mensagem foi aceita

PrecoControler — @PutMapping /preco

Mesma estrutura do EstoqueControler, mas opera sobre a fila PRECO e o DTO PrecoDTO.

6. Endpoints REST

6.1 Tabela de Endpoints

Método	Rota	Body de Entrada	Resposta	Fila Destino
POST	/pedido	PedidoDTO (JSON)	200 + JSON {status, mensagem}	PEDIDO
PUT	/estoque	EstoqueDTO (JSON)	200 sem body	ESTOQUE
PUT	/preco	PrecoDTO (JSON)	200 sem body	PRECO

6.2 Detalhamento dos Endpoints

POST /pedido

Envia um pedido para a fila PEDIDO.

Body de entrada (JSON):

```
{ "id": "PED-001", "cliente": "Maria Silva", "valor": 149.90 }
```

Resposta (HTTP 200):

```
{ "status": "sucesso", "mensagem": "Pedido PED-001 enviado para a fila
com sucesso!" }
```

Exemplo com curl:

```
curl -X POST http://localhost:8087/pedido \
  -H "Content-Type: application/json" \
  -d '{"id":"PED-001","cliente":"Maria","valor":149.90}'
```

PUT /estoque

Body de entrada:

```
{ "codigoproduto": "PROD-42", "quantidade": 100 }
```

PUT /preco

Body de entrada:

```
{ "codigoproduto": "PROD-42", "preco": "29.99" }
```

7. Mensageria RabbitMQ

7.1 Conceitos Utilizados

Conceito	Valor no Projeto	O que é
Exchange	amq.direct	Ponto de entrada das mensagens no broker; decide para qual fila rotear
Tipo de troca	Direct	Roteia mensagens pela routing key exata (sem wildcard)
Routing Key	= nome da fila	Chave usada para combinar mensagem com fila; aqui é idêntica ao nome da fila destino
Fila ESTOQUE	durable=true	Armazena mensagens de ajuste de estoque
Fila PRECO	durable=true	Armazena mensagens de atualização de preço
Fila PEDIDO	durable=true	Armazena mensagens de novos pedidos
Binding	fila ← exchange via routing key	Regra que liga a fila ao exchange

7.2 Como o Roteamento Funciona

O projeto usa o exchange padrão amq.direct (já existe no RabbitMQ por padrão). O modo Direct significa que a mensagem só chega a uma fila se a routing key da mensagem for idêntica à routing key do binding.

Exemplo prático:

- Mensagem publicada com routing key = "PEDIDO"
- O exchange verifica: qual fila tem binding com routing key "PEDIDO"?
- Resposta: a fila PEDIDO. A mensagem é entregue a ela.
- As filas ESTOQUE e PRECO não recebem essa mensagem.

7.3 Durabilidade das Filas

Todas as filas são criadas com durable=true, o que significa que elas sobrevivem ao reinício do servidor RabbitMQ. As mensagens em filas duráveis também são persistidas em disco (se publicadas com modo persistente), garantindo que nenhum pedido seja perdido em caso de queda.

8. Frontend (index.html)

8.1 Visão Geral

O frontend é uma página HTML estática que serve como interface visual para testar o envio de pedidos. Ela se comunica com o backend via API Fetch (JavaScript nativo, sem frameworks).

8.2 Campos do Formulário

Campo	Tipo HTML	Mapeamento no Backend	Validação
id	text	PedidoDTO.id	Obrigatório (preenchimento manual)
cliente	text	PedidoDTO.cliente	Obrigatório
valor	number	PedidoDTO.valor	Número decimal positivo

8.3 Fluxo de Comunicação

- Usuário preenche os campos e clica em 'Enviar Pedido'
- JavaScript coleta os valores do formulário e monta um objeto JSON
- fetch('http://localhost:8087/pedido', { method: 'POST', body: JSON.stringify({...}) }) é chamado
- O backend processa, envia à fila e retorna { status: 'sucesso', mensagem: '...' }
- O JavaScript exibe o feedback na tela (mensagem de sucesso ou erro)
- O pedido enviado é adicionado a uma lista de histórico visível na página

Ponto de atenção — CORS:

Para que o navegador permita que a página HTML acesse a API em localhost:8087, é necessário configurar o CORS no Spring Boot (adicionar @CrossOrigin nos controllers ou uma configuração global de WebMvcConfigurer). Sem isso, o navegador bloqueará a requisição.

9. Como Executar o Projeto

Pré-requisitos

Ferramenta	Versão Mínima	Para que serve
Java JDK	17	Compilar e executar o código Java
Maven	3.8	Gerenciar dependências e fazer o build
Docker	Qualquer	Subir o RabbitMQ em container (alternativa: instalar localmente)

Passo a Passo

Passo 1 — Suba o RabbitMQ:

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-
management
```

Passo 2 — Clone o repositório e acesse a pasta:

```
git clone https://github.com/vexedray/estudosDocker.git
cd coelho
```

Passo 3 — Execute a aplicação com Maven:

```
./mvnw spring-boot:run
# Ou no Windows:
mvnw.cmd spring-boot:run
```

Passo 4 — Verifique que está funcionando:

- Acesse o painel RabbitMQ: http://localhost:15672 (insira_seu_login_aqui/insira_sua_senha_aqui) — as filas ESTOQUE, PRECO e PEDIDO devem aparecer
- Abra o index.html no navegador e envie um pedido pelo formulário
- No painel do RabbitMQ, vá em Queues > PEDIDO e veja a mensagem na fila

Passo 5 — Teste via curl (opcional):

```
curl -X POST http://localhost:8087/pedido \
  -H "Content-Type: application/json" \
  -d '{"id":"001","cliente":"Ana","valor":99.90}'

curl -X PUT http://localhost:8087/estoque \
  -H "Content-Type: application/json" \
```

```
  -d '{"codigoproduto":"P42","quantidade":50}'

curl -X PUT http://localhost:8087/preco \
  -H "Content-Type: application/json" \
  -d '{"codigoproduto":"P42","preco":"19.99"}'
```